

CORE COMPUTER SCIENCE

DBMS & SQL

A visual reference on databases, ER modeling, transactions, normalization & SQL

- › ER Diagrams & Entity Sets
- › Cardinality & Attributes
- › Keys & Functional Dependency
- › Normalization (1NF–BCNF)
- › Transactions & ACID
- › Schedules & Serializability
- › Relational Algebra & B-Trees
- › SQL — DDL, DML, Joins & Queries

01 Introduction to DBMS

Database: A database is a collection of related data which represents some aspect of the real world. A database system is designed to be built and populated with data for a certain task.

Database Management System (DBMS): Software for storing and retrieving users' data while considering appropriate security measures. It consists of a group of programs that manipulate the database. The DBMS accepts requests for data from an application and instructs the operating system to provide the specific data. In large systems, a DBMS helps users and other third-party software store and retrieve data.

Problems DBMS Solves (vs. File Processing)

1. Data Redundancy & Inconsistency

Same data duplicated across files, leading to mismatches.

2. Difficulty Accessing Data

No flexible way to query across files.

3. Data Isolation

Multiple files and formats scattered across the system.

4. Integrity Problems

No centralized way to enforce data-correctness rules.

5. Atomicity of Updates

Partial updates can leave data inconsistent after a failure.

6. Concurrent Access

Multiple users editing the same data can conflict.

7. Security Problems

No fine-grained control over who can see or edit what.

02 ER Diagrams & Entity Sets

- An **ER (Entity Relationship) diagram** is a conceptual model that gives the graphical representation of the logical structure of the database.
- It shows all the constraints and relationships that exist among the different components.
- An ER diagram is mainly composed of three components: **Entity Sets**, **Attributes** and **Relationship Sets**.

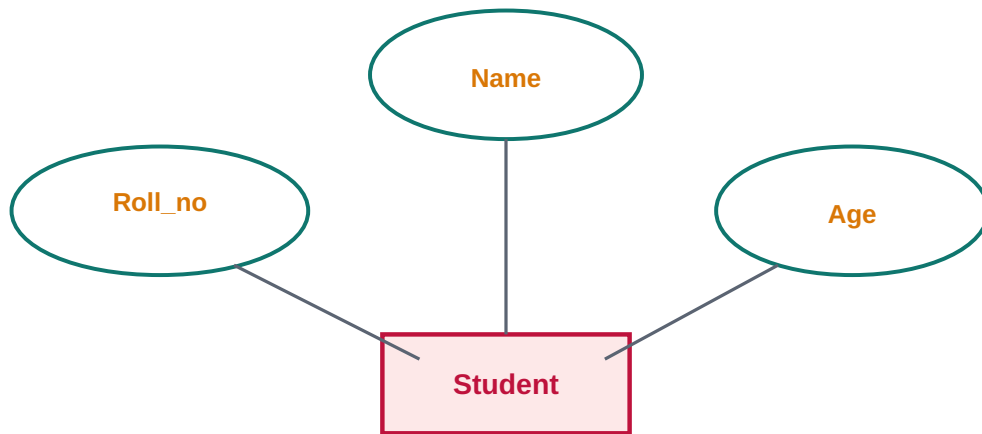
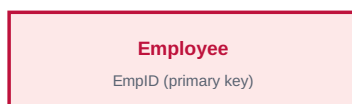


Fig 1 — ER diagram for the Student entity: Roll_no (underlined) is the primary key that identifies each entity uniquely.

Entity Set

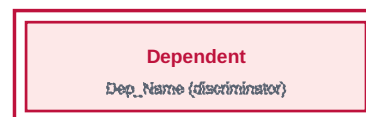
An entity set is a set of entities of the same type.

Strong Entity Set



Has a primary key — sufficient attributes to identify entities uniquely

Weak Entity Set



No primary key — only a partial key (discriminator, dashed underline)

Fig 2 — Strong entity sets have a full primary key; weak entity sets (double border) rely on a discriminator instead.

Strong Entity Set

Contains sufficient attributes to uniquely identify all its entities — a primary key exists, shown underlined.

Weak Entity Set

Lacks a full primary key; identified using a partial key called a **discriminator**, shown with a dashed underline.

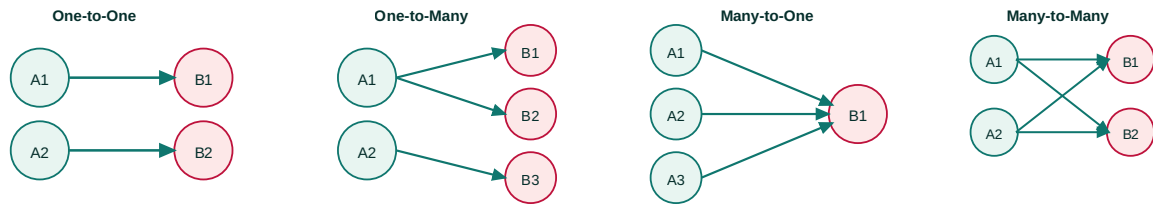
03 Relationships & Cardinality

A **relationship** is an association among several entities.

Type	Meaning
Unary	Only one entity set participates in the relationship.
Binary	Two entity sets participate in the relationship.
Ternary	Three entity sets participate in the relationship.

Cardinality Constraint

Defines the maximum number of relationship instances an entity can participate in.



Left set = A, Right set = B, in each pairing above

Fig 3 — The four cardinality constraints between entity sets A and B.

One-to-One

An entity in A relates to at most one entity in B, and vice versa.

One-to-Many

An entity in A relates to any number of entities in B; an entity in B relates to at most one in A.

Many-to-One

An entity in A relates to at most one entity in B; an entity in B relates to any number in A.

Many-to-Many

Entities on both sides may relate to any number of entities on the other side.

04 Attributes & Constraints

Attributes are the descriptive properties owned by each entity of an entity set.

Simple

Cannot be divided further. Example: Age.

Composite

Composed of several simple attributes. Example: Name, Address.

Multi-Valued

Can take more than one value for a given entity. Example: Mobile No, Email ID.

Derived

Can be derived from other attribute(s). Example: Age from Date of Birth.

Key Attribute — can identify an entity uniquely in an entity set. Example: Roll No.

Relational Constraints

Constraint	Rule
Domain	The value of an attribute must be an atomic value from its defined domain.
Tuple Uniqueness	All tuples must be unique within any relation.
Key	All primary key values must be unique and not null.
Entity Integrity	No attribute of the primary key can contain a null value.
Referential Integrity	Every foreign key value must exist in the referenced relation, or be null.

05 Keys

A **key** is a set of attributes that can identify each tuple uniquely in a relation.

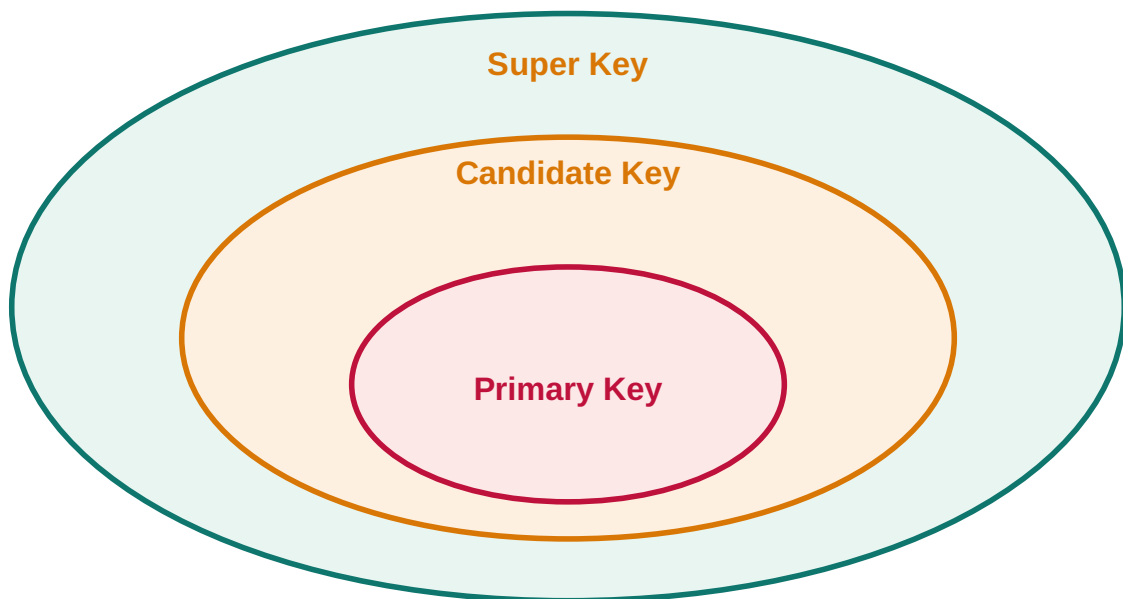


Fig 4 — Every primary key is a candidate key; every candidate key is a super key.

Super Key

Any set of attributes that can identify each tuple uniquely — may include extra, non-essential attributes.

Candidate Key

A minimal set of attribute(s) that can identify each tuple uniquely.

Primary Key

The candidate key chosen by the designer to represent the relation. Unique and NOT NULL.

Alternate Key

Candidate keys left unused after the primary key is chosen.

Foreign Key

An attribute whose values depend on another attribute in a different (referenced) relation.

Composite Key

A primary key made of multiple attributes rather than a single one.

UNIQUE KEY

Unique across all records in the table. Once assigned, its value cannot be changed — but it may hold a NULL value.

06 Functional Dependency & Decomposition

Closure of an Attribute Set: the set of all attributes that can be functionally determined from a given attribute set.

A functional dependency $\alpha \rightarrow \beta$ holds if two tuples with the same value of α also share the same value of β .

Trivial FD

$X \rightarrow Y$ is trivial if $Y \subseteq X$ (RHS is a subset of LHS).

Non-Trivial FD

$X \rightarrow Y$ is non-trivial if at least one attribute in Y is not part of X .

Decomposition of a Relation

Breaking a single relation into two or more sub-relations.

Lossless Decomposition

No information is lost; joining the sub-relations reproduces the original relation exactly: $R1 \bowtie R2 \bowtie \dots \bowtie Rn = R$.

Dependency Preservation

All functional dependencies of the original relation still hold across the sub-relations.

LOSSY JOIN DECOMPOSITION

Occurs when the join of the sub-relations does NOT reproduce the original relation: $R1 \bowtie R2 \bowtie \dots \bowtie Rn \supset R$.

07 Normalization

Database normalization makes a database consistent by reducing redundancies and ensuring data integrity through lossless decomposition.

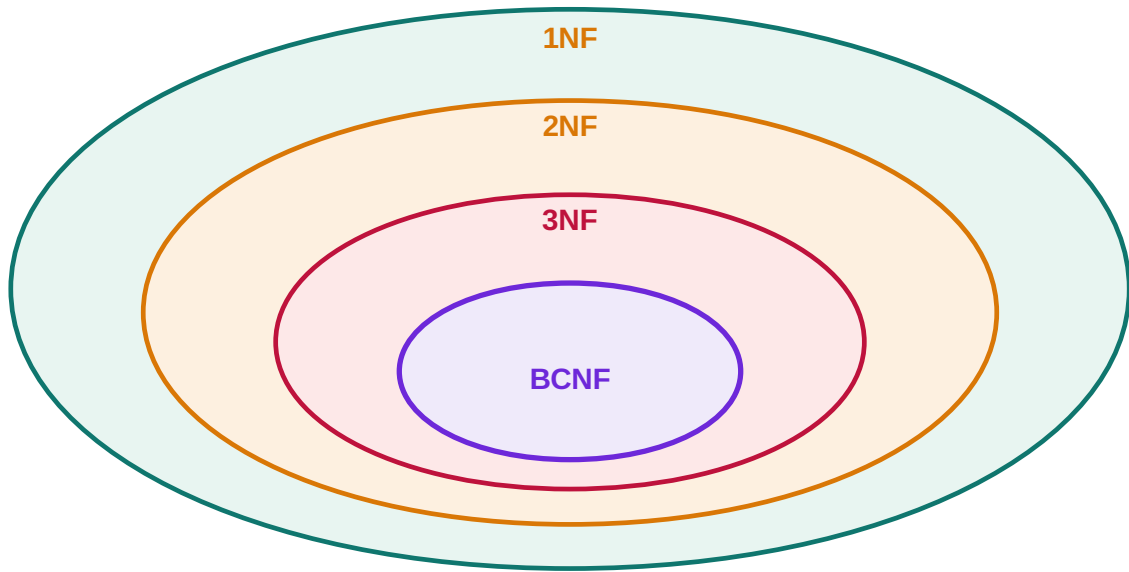


Fig 5 — Each normal form is a stricter subset of the one before it: $1NF \supset 2NF \supset 3NF \supset BCNF$.

Normal Forms

Form	Requirement
1NF	Every cell holds only an atomic value — each tuple's attribute is single-valued or null.
2NF	Already in 1NF, and no partial dependency exists ($A \rightarrow B$ where A is a subset of a candidate key and B is non-prime).
3NF	Already in 2NF, and no transitive dependency exists for non-prime attributes ($A \rightarrow B$ where A is not a super key and B is non-prime).
BCNF	Already in 3NF, and for every non-trivial FD ' $A \rightarrow B$ ', A must be a super key of the relation.

08 Transactions & ACID Properties

A **transaction** is a single logical unit of work formed by a set of operations. **Read(A)** reads a value into the memory buffer; **Write(A)** writes an updated value from the buffer back to the database.

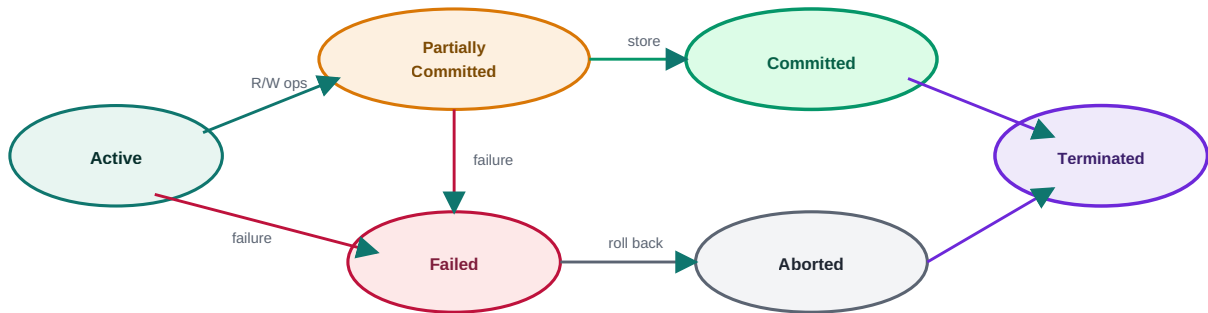


Fig 6 — Transaction lifecycle: Active → Partially Committed → Committed → Terminated, with Failed/Aborted as the error path.

State	Meaning
Active	First state — instructions are being executed; changes live only in the buffer.
Partially Committed	Last instruction executed, but changes are still only in the buffer, not the disk.
Committed	All changes successfully written to the database — fully committed.
Failed	A failure occurs during Active or Partially Committed state, halting execution.
Aborted	Reached after the transaction is fully rolled back following a failure.
Terminated	Final state — reached from either Committed or Aborted.

ACID Properties

Atomicity

The transaction happens completely, or not at all — never partially.

Consistency

Integrity constraints are maintained; the database stays consistent before and after.

Isolation

Concurrent transactions produce the same result as if run serially, one after another.

Durability

Committed changes are written to disk permanently and survive any failure.

09 Schedules & Serializability

The order in which operations of multiple transactions appear for execution is called a **schedule**.

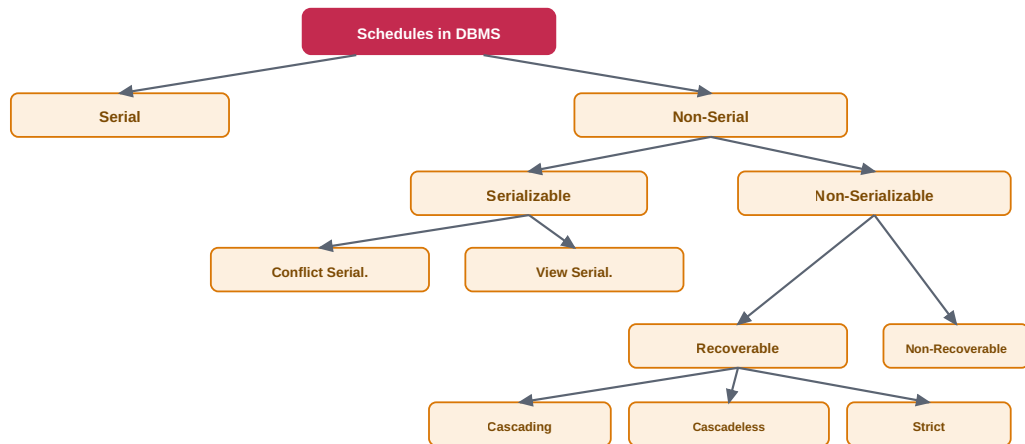


Fig 7 — Classification of schedules from serial/non-serial down to strict/cascadeless recoverable schedules.

Serial Schedules

Transactions execute one after the other with no overlap. Always Consistent, Recoverable, Cascadeless and Strict.

Non-Serial Schedules

Transactions execute concurrently, with operations interleaved. Not always Consistent, Recoverable, Cascadeless or Strict.

Serializability

Some non-serial schedules can cause inconsistency. **Serializability** identifies which non-serial schedules are correct and preserve consistency — a schedule equivalent to some serial schedule of the same 'n' transactions is called serializable, and is always Consistent, Recoverable, Cascadeless and Strict.

Conflict Serializable

Can be converted to a serial schedule by swapping non-conflicting operations.

View Serializable

Can be viewed as equivalent to some serial schedule.

Recoverability

Irrecoverable Schedule

A transaction reads an uncommitted value (dirty read) and commits before the source transaction commits.

Recoverable Schedule

A transaction's commit is delayed until the transaction it read a dirty value from has committed or rolled back.

Type	Rule
Cascading	Failure of one transaction forces several dependent transactions to roll back.
Cascadeless	A transaction can't read a data item until the last writer has committed or aborted.
Strict	A transaction can neither read nor write a data item until the last writer has committed or aborted.

10 Relational Algebra

Relational Algebra is a procedural query language — it takes a relation as input and produces a relation as output.

Basic Operators

Operator	Semantics
σ (Selection)	Selects rows based on a given condition.
π (Projection)	Projects (keeps) selected columns.
$\times$ (Cross Product)	Returns $m \times n$ rows, where m and n are the row counts of R_1 and R_2 .
$\cup$ (Union)	Tuples in R_1 or R_2 . Rows range from $\max(m,n)$ to $m+n$.
$-$ (Minus)	$R_1 - R_2$ returns tuples in R_1 but not R_2 . Rows range from $m-n$ to m .
ρ (Rename)	Renames a relation.

Extended Operators

Operator	Semantics
$\cap$ (Intersection)	Tuples in both R_1 and R_2 . Rows range from 0 to $\min(m,n)$.
$\bowtie_c$ (Conditional Join)	Cross product followed by a selection on a given condition.
$\bowtie$ (Equi Join)	Conditional join using only equality conditions.
$\bowtie$ (Natural Join)	Equality on common attributes; duplicate attributes removed automatically.
$\bowtie_l$ (Left Outer Join)	All tuples of R kept; unmatched S attributes become NULL.
$\bowtie_r$ (Right Outer Join)	All tuples of S kept; unmatched R attributes become NULL.
$\bowtie$ (Full Outer Join)	All tuples of both R and S kept; unmatched attributes on either side become NULL.
$\div$ (Division)	A/B returns tuples in A associated with every tuple of B .

NOTE

Natural Join is equivalent to Cross Product when R and S share no common attribute; the natural join of R with itself just returns R .

11 File Structures & B-Trees

Primary Index

An ordered file with fixed-length records: a primary-key field and a pointer to the data block. Access cost $\approx \log_2(B_i) + 1$, where B_i = number of index blocks.

Clustering Index

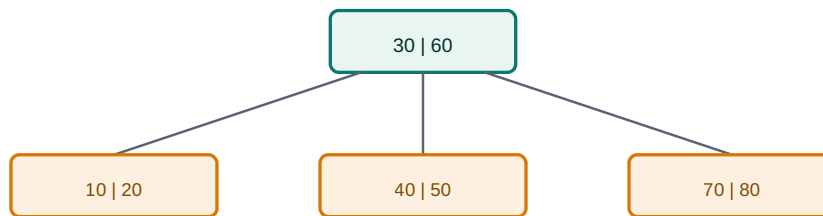
Built on a data file whose records are physically ordered by a non-key clustering field.

Secondary Index

Provides an additional access path on a file that already has a primary access method.

B-Trees

At every level, each node holds keys and data pointers; a data pointer points to a block or record.



Order P = maximum children per node; root has $2..P$ children

Fig 8 — A B-Tree of order P : internal nodes hold between $\lceil P/2 \rceil - 1$ and $P - 1$ keys.

B-Tree Rules

Root: 2 to P children. Internal node: $\lceil P/2 \rceil$ to P children, and $\lceil P/2 \rceil - 1$ to $P - 1$ keys.

B+ Tree

Leaf and non-leaf nodes have different structures and orders. Non-leaf nodes have no record pointers, so depth — and search time — is smaller than a B-Tree.

12 SQL Language Categories

DDL — Data Definition Language

Deals with database schemas. **CREATE** a database/object, **ALTER** its structure, **DROP** it, **TRUNCATE** all records (and space), **RENAME** an object.

DML — Data Manipulation Language

Manipulates data. **SELECT** retrieves, **INSERT** adds, **UPDATE** modifies, **DELETE** removes, **MERGE** performs an upsert.

DCL — Data Control Language

Manages access rights. **GRANT** gives privileges, **REVOKE** withdraws them.

TCL — Transaction Control Language

Controls transactions. **COMMIT** saves changes, **ROLLBACK** undoes them, **SAVEPOINT** marks a rollback point.

13 SQL — Core Queries

SELECT & SELECT DISTINCT

SYNTAX

```
SELECT column1, column2, ... FROM table_name;
SELECT * FROM table_name;           -- all columns
SELECT DISTINCT column1 FROM table_name; -- unique values only
```

EXAMPLE

```
SELECT CustomerName, City FROM Customers;
SELECT DISTINCT Country FROM Customers;
```

WHERE, AND / OR / NOT

The WHERE clause filters records; it can combine multiple conditions using AND, OR, and NOT.

Operator	Meaning
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<> (or !=)	Not equal

EXAMPLE

```
SELECT * FROM Customers WHERE Country='Mexico';
SELECT * FROM Customers
WHERE Country='Germany' AND (City='Berlin' OR City='München');
```

ORDER BY

Sorts the result set — ascending by default, or descending with **DESC**.

EXAMPLE

```
SELECT * FROM Customers ORDER BY Country;
SELECT * FROM Customers ORDER BY Country ASC, CustomerName DESC;
```

INSERT INTO

SYNTAX & EXAMPLE

```
INSERT INTO table_name (col1, col2, ...) VALUES (val1, val2, ...);
```

```
INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode, Country)
VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway');
```

NULL Values

Comparison operators (=, <, <>) cannot test for NULL — use **IS NULL** / **IS NOT NULL** instead.

EXAMPLE

```
SELECT CustomerName, ContactName, Address
FROM Customers WHERE Address IS NULL;
```

UPDATE & DELETE

EXAMPLE

```
UPDATE Customers SET ContactName = 'Alfred Schmidt', City = 'Frankfurt'
WHERE CustomerID = 1;
```

```
DELETE FROM Customers WHERE CustomerName='Alfreds Futterkiste';
DELETE FROM Customers; -- deletes all rows, keeps table structure
```

SELECT TOP / LIMIT

EQUIVALENT FORMS ACROSS DATABASES

```
SELECT TOP 3 * FROM Customers;
SELECT * FROM Customers LIMIT 3;
SELECT * FROM Customers ORDER BY ... FETCH FIRST 3 ROWS ONLY;
SELECT * FROM Customers WHERE ROWNUM <= 3;
```

14 Aggregate Functions & Pattern Matching

Function	Returns
MIN()	Smallest value in the selected column.
MAX()	Largest value in the selected column.
COUNT()	Number of rows matching a criterion.
AVG()	Average value of a numeric column.
SUM()	Total sum of a numeric column.

EXAMPLE

```
SELECT MIN(Price) AS SmallestPrice FROM Products;
SELECT COUNT(ProductID) FROM Products;
SELECT AVG(Price) FROM Products;
```

LIKE Operator

% matches zero, one or multiple characters; _ matches exactly one character.

Pattern	Matches
LIKE 'a%'	Starts with "a"
LIKE '%a'	Ends with "a"
LIKE '%or%'	Contains "or" anywhere
LIKE '_r%'	"r" in the second position
LIKE 'a_%'	Starts with "a", at least 2 characters
LIKE 'a__%'	Starts with "a", at least 3 characters
LIKE 'a%o'	Starts with "a" and ends with "o"

IN & BETWEEN

IN is shorthand for multiple OR conditions. **BETWEEN** selects an inclusive range of numbers, text, or dates.

EXAMPLE

```
SELECT * FROM Customers WHERE Country IN ('Germany', 'France', 'UK');
SELECT * FROM Products WHERE Price BETWEEN 10 AND 20;
```

15 SQL Joins

A **JOIN** combines rows from two or more tables based on a related column.



Left circle = table1 · Right circle = table2 · Amber = rows included in the result

Fig 9 — The four core SQL joins, shown by which rows of each table make it into the result.

Join	Behaviour
INNER JOIN	Only rows with matching values in both tables.
LEFT (OUTER) JOIN	All rows from the left table, plus matches from the right (NULL where no match).
RIGHT (OUTER) JOIN	All rows from the right table, plus matches from the left (NULL where no match).
FULL (OUTER) JOIN	All rows from both tables; unmatched sides filled with NULL.

EXAMPLE — INNER & LEFT JOIN

```
SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
INNER JOIN Customers ON Orders.CustomerID = Customers.CustomerID;

SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
ORDER BY Customers.CustomerName;
```

UNION

Combines the result sets of two or more **SELECT** statements — same number and order of columns with similar data types required. Returns distinct values by default; use **UNION ALL** to keep duplicates.

EXAMPLE

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers ORDER BY City;
```

16 GROUP BY & HAVING

GROUP BY groups rows with the same values into summary rows, usually paired with aggregate functions (COUNT, MAX, MIN, SUM, AVG).

EXAMPLE

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
ORDER BY COUNT(CustomerID) DESC;
```

HAVING VS. WHERE

HAVING exists because **WHERE** cannot be used with aggregate functions. **WHERE** is evaluated first, before **GROUP BY** / **HAVING**.

EXAMPLE

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5;
```

17 Database & Table Statements

DATABASE

```
CREATE DATABASE databasename;
DROP DATABASE databasename;
```

TABLE

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    ...
);
DROP TABLE table_name;
TRUNCATE TABLE table_name;  -- deletes data, keeps structure
```

ALTERING A TABLE

```
ALTER TABLE Customers ADD Email varchar(255);
ALTER TABLE Customers DROP COLUMN Email;
ALTER TABLE Persons ALTER COLUMN DateOfBirth year;
```